

ROBOTICS · LESSON 5

The Loop Closes

Sense, decide, act, sense again — the rover drives until it sees an obstacle within a safe distance, then stops and turns. The loop closes.

ROBO-005 · ~45 MIN · INTERMEDIATE

1. The one idea

Close the loop. The rover senses the world and decides for itself: drive forward until the ultrasonic sees an obstacle inside a safe distance, then stop and turn. **Sense** → **decide** → **act** → **sense again**, with no human in the loop.

2. Why it matters

Lessons 1–4 were open loop: command out, no check. A *closed* loop feeds the result back — the robot acts, senses the consequence, and acts again. That feedback is the line between a machine running a script and a robot coping with a world it didn't fully predict. And the decision here is a real safety one — *don't drive into things* — so the safe distance is a genuine **guard**: the Lesson 2 clamp, now governing a moving robot instead of a servo.

3. Parts & wiring

Nothing new — this is integration. Take the Lesson 4 rover and add the Lesson 3 ultrasonic sensor (with its ECHO divider) facing forward.

- The Lesson 4 rover: L298N + two motors + motor battery, on GPIO25/26/27/14/4/13
- The Lesson 3 sensor: TRIG on GPIO23, ECHO on GPIO22 through its divider
- One shared ground across the ESP32, the L298N, and the sensor

4. Predict (do this before uploading)

With a safe distance of 20 cm:

- As the rover nears a wall, at what point does it stop?
- After it stops, what should it do — and for how long?
- If the sensor returns a **bad reading** (out of range, a missed echo), should the rover stop or keep going? Decide *now*, before the code tells you.

5. Build it — read, decide, act, repeat

```
// Reuses readDistanceCm() from Lesson 3 and drive() from Lesson 4.
const int SAFE_CM = 20;    // the guard: do not approach closer than this

void loop() {
  long cm = readDistanceCm();
  bool blocked = (cm < 0) || (cm < SAFE_CM);    // a bad reading counts as blocked

  if (blocked) {
    drive(0, 0, 0);    // stop at the guard
    delay(150);
    drive(+1, -1, 200);    // turn away
    delay(400);
  } else {
    drive(+1, +1, 200);    // clear ahead: go
  }

  Serial.printf("dist=%3ld  %s\n", cm, blocked ? "BLOCKED → turn" : "clear → go");
  delay(60);
}
```

The shape is `read → decide → act`, then **loop back and read again**. The decision is a guard — `cm < SAFE_CM` — the same clamp logic from Lesson 2, now stopping a moving robot rather than a servo. And `cm < 0` (a bad reading) counts as *blocked*: that is the **fail-safe** — when the robot can't see, it stops rather than charges. A guard that trusts a missing reading isn't a guard.

6. Test against your prediction

Set the rover down and let it drive at a wall. It should stop about 20 cm out and turn away. Put your hand in front and it backs off. Now the real test: cover the sensor or aim it at empty space so it misreads — does the rover **stop** (good) or **lurch** (bad)? That moment is where the fail-safe earns its line of code.

7. What goes wrong (pre-loaded debugging)

- **It drives into the wall** → the loop is too slow or `SAFE_CM` is too small for the speed; it can't stop in time. Slow down or stop earlier — stopping distance is real even at this scale.

- **It stops and spins forever** → it turned to face another obstacle, or `SAFE_CM` is too large for the room. Shorten the turn or the distance.
- **It charges through obstacles now and then** → ultrasonic noise; one bad ping reads "clear." Require two clear reads in a row before driving (debounce the decision).
- **It lurches on an out-of-range reading** → you treated a bad reading as "clear." Treat it as blocked — fail safe.

8. Checkpoint

The loop is closed. The rover senses, decides under a safety guard, acts, and senses again — autonomy, on your bench. You have now built every piece of the basic robot: **actuate** (L1), **guard the command** (L2), **sense the world** (L3), **locomote** (L4), and **close the loop** (L5). Everything past here — better sensors, more joints, real control theory, mapping — is this same loop, repeated and refined.

9. Push further

- **PID speed control with wheel encoders** — the motor checks whether it actually got where it was told. This is the control-theory lesson Lesson 1 deliberately deferred; you've earned it now.
- **Proportional steering** — turn harder the closer the obstacle, instead of a fixed stop-and-turn.
- **A field of view** — mount the sensor on a servo (Lessons 1 + 3) and scan, so the rover chooses the most open direction instead of just turning away.